Information Technology Course
Module Software Engineering
by Damir Dobric

ML25/26-11 Implement Reminder Agent

KavyaSree, Thabjul -1633324 (thabjul.sree@stud.fra-uas.de)
Bavana Jadala-1629828 (bavana.jadala@stud.fra-uas.de)
Aishwarya Rajendra Sawant-1629688 (aishwarya.sawant@stud.fra-uas.de)

***Abstract* — Personal information management tools consistently fail at both capture and retrieval. They demand structured input when saving and exact keyword recall when searching, yet people routinely cannot reconstruct their original search terms. This paper presents Reminder Agent, a conversational AI system built in C# on the Microsoft Agent Framework that accepts free-form natural language for both saving and retrieving personal assets such as restaurants, books, travel ideas, and experiences. The system routes each user request through GPT-4o-mini to one of seven typed tool functions and stores assets in lightweight CSV and JSON files with no database required. Retrieval adapts to the nature of the request. When the user provides a concrete filter such as a date, location, or category, structured metadata filtering is applied. When the request is descriptive or context-based, cosine similarity over 1,536-dimensional OpenAI embeddings is used. A two-tier test suite covering unit and integration levels, run against real files on disk, verified correct behaviour across storage round-trips, semantic edge cases, concurrent writes, and duplicate detection. The approach demonstrates that file-based persistence with atomic write guarantees is a viable alternative to a database for personal-use scenarios, with the principal limitations being linear scalability of the cosine scan and single-user storage isolation.**

***Keywords*—Natural Language Processing, Personal Information Management, Conversational AI, Microsoft Agent Framework, Hybrid Retrieval, Semantic Search, Vector Embeddings, Cosine Similarity, Duplicate Detection, File-based Storage, C#**

I. INTRODUCTION

Personal information management tools fail at both ends of the task. At capture time they demand categories, tags, and structured fields; at retrieval time they require the user to recall the exact words used when saving. Studies show that people consistently fail to retrieve things they know they saved, not because the item is missing but because they cannot reconstruct the original search terms [1]. The gap between how people save and how they later search is a design flaw that structured forms and keyword indices cannot fix.

Popular tools such as note-taking applications and bookmark managers partially address this with tagging and full-text search, but both still depend on the user supplying structure at capture time. Voice assistants lower the capture barrier but require exact commands. None of these approaches helps a user who only remembers the feeling of an experience or a vague description of a place visited, and none adapts the retrieval strategy to the type of query being asked.

Large language models change what is possible. A system that accepts plain natural language at both capture and retrieval time removes both friction points simultaneously [2]. This paper investigates whether LLM-based intent routing can reliably handle personal asset operations from free-form sentences, whether combining metadata filtering with semantic vector search outperforms either strategy alone, and whether file-based atomic storage can substitute for a database in the personal-use scenario without sacrificing consistency.

Reminder Agent was built at Frankfurt University of Applied Sciences to answer these questions. The main contributions are a working conversational agent for personal asset management requiring no structured input, a hybrid retrieval design that routes to metadata filtering or cosine similarity based on the nature of the request, a file-based persistence layer with atomic write guarantees, and an integration test suite that exercises the full stack against real on-disk files.

II. LITERATURE REVIEW

A. Conversational AI and Intent Routing

With the ReAct framework, the use of LLMs as intent routers over typed tool functions was formalised [3]. The core idea is that the model reasons about what to do, calls a tool, reads the result, and reasons again rather than generating a response in one shot. This contrasts with earlier task-oriented dialogue systems, which required hand-crafted state machines and slot-filling grammars to interpret user requests. By replacing the state machine with a language model, the system handles arbitrary phrasings of the same intent without additional engineering effort and without needing to enumerate all possible input variations in advance. MAF brings this pattern into the .NET ecosystem through `IChatClient`, `AIFunctionFactory`, and `UseFunctionInvocation()` middleware [4]. In Reminder Agent, each of the seven tool functions is registered with a `[Description]` attribute that the LLM reads at routing time to determine both which method to invoke and what arguments to extract from the free-form sentence, so the routing contract lives directly alongside the code that implements it.

B. Retrieval-Augmented Generation

Lewis et al. [2] introduced RAG as a mechanism for grounding LLM output in retrieved factual context, significantly

reducing hallucination on knowledge-intensive tasks. The core insight is that a language model should retrieve relevant context at query time rather than relying on parametric memory. In Reminder Agent, retrieval and generation are kept as two separate steps. The tool fetches and filters assets, builds a context string, and passes it to a second LLM call that produces readable prose. This two-stage design separates retrieval correctness, which is verified through deterministic tests, from generation quality.

C. Vector Embeddings for Semantic Search

Devlin et al. [5] showed that context-aware token representations significantly outperform keyword matching for semantic similarity, representing a fundamental departure from sparse TF-IDF representations that treat each word independently and cannot capture synonymy or paraphrase. Reimers and Gurevych [6] extended this to sentence-level embeddings using Siamese network fine-tuning, making it practical to compare full sentences by cosine distance. OpenAI's text-embedding-3-small produces 1,536-dimensional vectors optimised for similarity and retrieval [7]. Reminder Agent constructs a composite embedding string per asset by concatenating name, category, experience, user input, and tags before encoding, ensuring that the resulting vector captures the full semantic context of the asset rather than any single field in isolation. This means a search for "cosy Italian place" can surface an asset whose name is "Vapiano" if the category is "restaurant" and the experience field mentions warmth or atmosphere, even though none of those words appear in the query verbatim.

D. Adaptive Retrieval Routing

Jeong et al. [8] showed that selecting retrieval strategy based on query complexity outperforms a single fixed approach across all query types. Simple factual queries benefit from direct retrieval while complex or ambiguous queries benefit from more elaborate pipelines involving re-ranking or multi-step reasoning. Reminder Agent applies a simpler version of this principle at the tool routing level. When the user's request contains a concrete filter such as a date range, a category name, or a location, the system uses structured metadata filtering, which is both faster and more precise for that class of input. When no such filter is present and the request is descriptive or context-based, the system generates a query embedding and performs cosine similarity search over the stored asset embeddings. The routing decision is made by the LLM based on the content of the request, without requiring the user to specify which mode they want or to understand the distinction.

E. Flat-File Persistence and Crash Safety

Skeen and Stonebraker [10] established that durability guarantees require either write-ahead logging or atomic rename operations. Reminder Agent applies the atomic rename pattern to both its CSV and JSON stores, using a `SemaphoreSlim(1,1)` concurrency gate and `File.Replace()` for the atomic swap, providing equivalent safety without a full database engine. The choice of flat-file storage also serves a practical purpose beyond simplicity: the CSV file can be opened directly in Excel, inspected, and even manually edited without any tooling, making it accessible to a supervisor who wants to verify the stored data without installing additional software.

F. Comparison with Existing Personal Information Tools

Existing personal information tools occupy distinct positions on the capture and retrieval trade-off spectrum. Google Keep and Apple Reminders offer frictionless capture but retrieval is limited to keyword search and manual label browsing, so neither tool can surface an asset when the user only remembers a feeling or a vague description. Notion provides rich structured storage and full-text search but places the highest capture burden on the user because effective retrieval requires consistent use of tags, databases, and page hierarchies that most users abandon over time. Dedicated vector database systems such as Pinecone or Weaviate provide powerful semantic search but require cloud infrastructure, API keys, schema design, and ongoing maintenance that are disproportionate for personal use.

Reminder Agent occupies a different position. It accepts unstructured input like a voice assistant, provides semantic retrieval like a vector database, and deploys as a single folder of flat files like a spreadsheet. No existing tool combines all three properties. The trade-off is linear scalability of the cosine scan, which would require an approximate nearest-neighbour index at large scale, and single-user storage isolation, which a shared database would provide natively.

III. METHODOLOGY

A. System Architecture

Reminder Agent is structured as a four-tier .NET application, as illustrated in Fig. 1. The system has no graphical user interface; all interaction takes place through the command line.

The **Presentation** tier is a console-based Read-Eval-Print Loop implemented in `Program.cs`. On each iteration the loop calls `Console.ReadLine()` to read a line of natural language text from standard input, appends it as a `ChatMessage` with role *user* to a `List<ChatMessage>` conversation history, and passes the full history to the Orchestration tier. When the agent returns a response, the loop writes it to standard output via `Console.WriteLine` and waits for the next input. The conversation history is kept in memory for the lifetime of the process, giving the model access to the full prior context on every turn. The user sees a plain terminal prompt with no menus, buttons, or windows.

The **Orchestration** tier uses GPT-4o-mini via MAF's `IChatClient` to select the appropriate tool and extract structured arguments from the user's sentence. The **Tool** tier exposes seven public methods of `ReminderTools` as `AIFunctions` that execute business logic. The **Infrastructure** tier comprises `CsvStorageProvider`, `JsonEmbeddingStore`, `EmbeddingService`, `SimilarityService`, and `FileLogger`, all registered as singletons via .NET dependency injection. The Orchestration

tier makes a second GPT-4o-mini call to convert raw tool output into natural-language prose before it reaches the user.

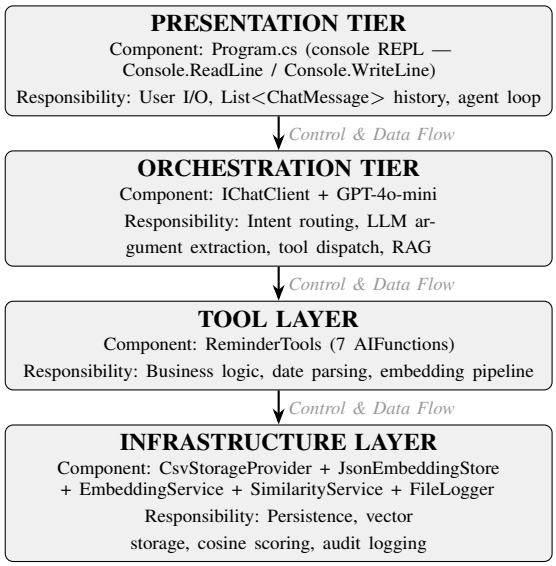

Fig. 1. Reminder Agent four-tier architecture. The Presentation tier is a plain console REPL with no graphical interface.

B. LLM Argument Extraction

As illustrated in Fig. 2, the first step after the user types a sentence is LLM Argument Extraction. This is the step where GPT-4o-mini reads the user's free-form input and converts it into a set of named, typed fields that the rest of the pipeline can work with. Without this step, the system would have no way of knowing what to store — it would receive a raw sentence and have no structured data to embed, check for duplicates, or write to the CSV file. The remaining boxes in the figure (Embedding Generation, Duplicate Detection, the two storage writes, Retrieval, and RAG Prose Generation) all depend on the output of this first step.

When the user types a natural language sentence, GPT-4o-mini performs argument extraction using the OpenAI function-calling mechanism before any storage or retrieval operation takes place. Each method in `ReminderTools` is annotated with `[Description(...)]` attributes at both the method level and each individual parameter. MAF serialises these annotations into a JSON schema and transmits it to GPT-4o-mini alongside the full conversation history. The model reads the schema, inspects the user's sentence, and emits a structured JSON object containing only the parameter fields it can infer from the input. MAF then deserialises this object directly into the C# method's parameter list. No manual string parsing, regular expressions, or hardcoded keyword matching are involved at any stage.

The parameters that `CreateAsset` can receive through this mechanism are: `name` (the asset title), `category` (e.g. Restaurant, Book, Travel), `tags` (a list of descriptive labels), `eventDate` (an absolute or relative date expression passed as a string), `city`, `country`, and `region` (location fields), `userInput` (the verbatim sentence the user typed), and `userExperience` (a free-text description of the experience). Location fields are inferred from geographic references embedded in the sentence: the input "Vapiano in Frankfurt" causes the model to populate `city=Frankfurt`, `country=Germany`, and `region=Hesse` without any explicit field label being provided by the user. Similarly, "visited the Colosseum" resolves to `city=Rome`, `country=Italy`, `region=Lazio`. The `eventDate` parameter accepts relative expressions such as "next month" or "last summer" verbatim; these strings are later resolved to concrete date ranges by the `ResolveRelativeDate` helper in the Tool tier.

The system prompt additionally provides worked inference examples for location extraction because parameter-level `[Description]` annotations alone were insufficient to produce consistent geographic field population during testing. The worked examples were added specifically to address this gap without changing the tool signatures or adding post-processing code. The same function-calling mechanism applies to all six remaining tool methods, each carrying its own schema so the model selects the correct tool and populates only the parameters relevant to that tool from the user's sentence. As illustrated in Fig. 2, the full lifecycle of an asset proceeds from argument extraction through embedding generation and duplicate detection to dual-store persistence, retrieval, and RAG prose generation.

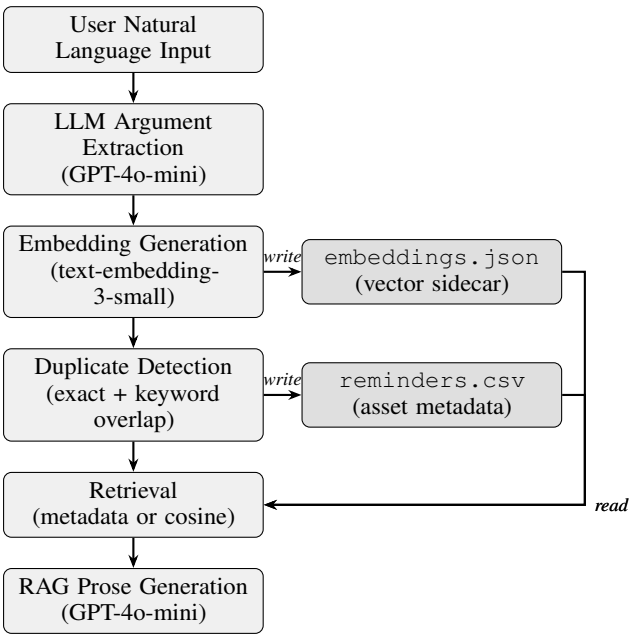

Fig. 2. Asset lifecycle from natural language input through LLM argument extraction, embedding, duplicate detection, dual-store persistence, to retrieval and RAG prose generation.

C. Tool Function Design

The system exposes seven tool functions that cover the full lifecycle of a personal asset. Each is registered with the MAF orchestration layer as an `AIFunction` via `AIFunc-`

tionFactory.Create. The LLM selects among them based on the intent it infers from the user's message and the [Description] attributes attached to each method. The following descriptions explain each tool in terms of its purpose, accepted parameters, routing trigger, and return behaviour. The [Description] attributes in the source code serve a distinct purpose and are written for LLM consumption.

CreateAsset is the primary data ingestion function. It receives the nine fields described in Section III-B, runs the full sequential ingestion pipeline (argument extraction, date resolution, embedding generation, duplicate detection, JSON write, CSV write), and persists the result to both storage files. The LLM invokes this function whenever the user's sentence describes something to be remembered or saved, such as a restaurant visited, a book finished, or a future travel plan.

SearchAssets performs structured retrieval. It accepts optional filters for category, a dateRange string, a location name, and a topK count. The function applies these filters against the metadata in reminders.csv and, when no metadata match is sufficient, falls back to cosine similarity over embeddings with threshold $\tau = 0.75$. The LLM invokes this function when the user's request contains at least one concrete constraint such as a specific city, time period, or category name.

GetReminders is a timeline-scoped listing function. It accepts a state parameter whose value is one of *Past*, *Present*, or *Future*. Rather than relying on a stored field, it dynamically computes each asset's timeline classification at query time by comparing EventDate against the current system clock, ensuring records migrate automatically from *Present* to *Past* once their date has passed without requiring data migration or background jobs. The LLM invokes this function when the user asks for reminders grouped by time horizon.

SearchRemindersSemantic handles open-ended or feeling-based queries for which no concrete filter is available. It accepts a free-text query string and a topK count, generates a query embedding using text-embedding-3-small, and ranks stored assets by descending cosine similarity with threshold $\tau = 0.25$. The lower threshold compared to SearchAssets is intentional: vague queries such as "that mountain place I liked" may produce moderate similarity scores, and a higher cutoff would return no results. The LLM invokes this function when the request is descriptive or experiential rather than filter-based.

AttachPhoto associates a photo file reference with an existing asset identified by assetName. It validates the file extension against a permitted list before writing, and updates only the target asset row in reminders.csv without touching any sibling records. The LLM invokes this function when the user asks to add or link an image to something already saved.

ListAssets returns all stored assets, optionally filtered by a category string. It is invoked when the user wants a full inventory view rather than a targeted search.

GetAssetDetails returns the complete field set for a single named asset, including all metadata, location details, tags, and photo references. It is invoked when the user asks for everything known about a specific item by name.

As summarised in Table I, each of the seven tool functions has a distinct set of key parameters and a specific trigger condition that determines when the LLM invokes it.

TABLE I
TOOL FUNCTIONS AND ROUTING CONDITIONS

Tool Function	Key Parameters	Trigger Condition
CreateAsset	name, category, userInput, eventDate, city, country, region, tags	New asset save request
SearchAssets	query, category, dateRange, location, topK	Query with concrete filter
GetReminders	state (Past / Present / Future)	Timeline-scoped listing
SearchReminders-Semantic	query, topK	Descriptive or feeling-based query
AttachPhoto	assetName, photo-Reference	Photo attachment request
ListAssets	category (optional)	Full inventory view
GetAssetDetails	assetName	Detailed single-asset lookup

D. Data Model and Project Structure

The system uses a single *Asset* entity to represent all reminders. Each asset is uniquely identified by a GUID and includes name, category, and tags for classification. Temporal information is captured through a TimelineState (*Past*, *Present*, or *Future*), an optional EventDate, and a creation timestamp. A flexible metadata dictionary (Dictionary<string, string>) holds supplementary facts about the asset, and UserInput and UserExperience are also retained, along with optional photo references. Semantic embeddings are stored separately in embeddings.json and reattached during data loading using the shared GUID key.

Two fields that are easy to confuse are tags and metadata, so it is worth explaining how they differ and how the LLM decides which words go where. Tags are short, searchable labels that describe the nature or theme of the asset, for example "summer", "travel", or "italian food". They are stored as a semicolon-separated list and are included in the composite embedding string, so they directly influence semantic search results. Metadata is a free-form dictionary for supplementary facts that do not fit any named field, such as a phone number, a price, or an author name. The [Description] attribute on the tags parameter explicitly instructs the model to use short categorical words, while the [Description] on the metadata parameter instructs it to capture supplementary factual details such as a phone number, a price, or an author name. A word like "summer" becomes a tag, while "Author: Robert C. Martin" becomes a metadata entry. We noticed during testing that the model does not always get this distinction right. For example, it occasionally places

a descriptive word that should be a tag into the metadata field instead. This is not a bug in the code it is a limitation of how LLM function calling works. The model reads the `[Description]` annotation and makes its best guess based on what the user typed. If the user's sentence is ambiguous, the model may assign a value to the wrong field. Research on LLM function calling has shown that the quality of argument extraction depends directly on how precisely the parameter descriptions are written [9].

The `TimelineState` field is not stored as a fixed value. Instead it is computed dynamically at query time by comparing the asset's `EventDate` against the current system clock. This means a restaurant visit saved as *Present* automatically appears as *Past* once its date has passed, without any update to the stored record and without any background process or scheduled job. The `GetReminders` tool relies on this dynamic computation to always return an accurate timeline view regardless of how long ago the asset was created.

The solution is organised into five folders. The `Domain` folder is named `Domain` rather than `Entities` because it contains only one entity, `Asset.cs`, and creating a folder called `Entities` for a single class would add structure without adding clarity. The name `Domain` better signals the purpose of the folder: it holds the central concept that the entire system is built around. Everything else in the project the interfaces, the infrastructure, the tools exists to serve or operate on this one entity. Naming the folder `Domain` makes that centrality explicit to anyone reading the codebase for the first time. The `Domain` folder therefore contains only `Asset.cs` the single entity that models all personal reminders in the system. The `Interfaces` folder holds the four behavioural contracts: `IStorageProvider`, `IEmbeddingStore`, `IEmbeddingService`, and `ISimilarityService`. Keeping interfaces in their own folder means that any layer of the application can depend on a contract without pulling in a data class or an implementation. The `Infrastructure` folder contains the concrete implementations of those interfaces: `CsvStorageProvider`, `JsonEmbeddingStore`, `EmbeddingService`, `SimilarityService`, and `FileLogger`. The `Tools` folder contains `ReminderTools.cs` with its seven `AIFunction` methods. The `Photos` folder is the default landing directory for photo references attached to assets. `Program.cs` at the root acts as the single composition root that wires all components together through the .NET Generic Host DI container.

E. Storage Design

Reminder data is stored in a CSV file (`reminders.csv`) within a dedicated `UserData` directory whose path is resolved at startup by walking up the directory tree until a `.csproj` file is found, then appending `UserData` to that location. Before saving, the system checks whether the file is locked by an external application and provides a user-friendly error message if so. A backup copy is created after each successful write to allow recovery from a failed subsequent write. Embedding vectors are stored separately in a JSON sidecar to avoid complications with large numerical arrays in CSV format. During retrieval, embeddings are loaded and merged with assets using their GUID keys.

The CSV file uses CsvHelper with a custom `AssetMap` class that handles serialisation of complex fields. Multi-value fields such as Tags and PhotoRefs are serialised as pipe-delimited strings within a single CSV cell, and the Metadata dictionary is serialised as a JSON fragment embedded in its own column. This approach keeps the file human-readable in Excel while still preserving structured data that standard CSV parsers cannot represent natively. Dates are written in unambiguous `DD-MM-YYYY` format to prevent silent misreads that would occur with slash-delimited formats when the day value is twelve or less.

1) Concurrency control: Write operations are serialised using a `SemaphoreSlim(1,1)` gate that allows at most one writer at a time. This prevents interleaved writes from corrupting either the CSV or the JSON store without requiring a database lock manager. The semaphore is acquired before any file is opened and released only after the atomic replace completes, ensuring that a second concurrent write waits until the first is fully committed.

2) Atomic write pattern: Every write, to both `reminders.csv` and `embeddings.json`, follows the same pattern. The new content is first written to a temporary file in the same directory, then `File.Replace()` atomically swaps it over the target. Because rename operations within the same filesystem are atomic at the OS level, a mid-write crash can only result in either the original file being intact or the new file being fully committed. Partial writes are impossible. The `SemaphoreSlim` gate and atomic rename together guarantee that the stores are never left in an inconsistent state, even under concurrent access or unexpected termination.

F. Sequential Ingestion Pipeline

`CreateAsset` executes six steps in strict order with no shared mutable state between steps. Step 1 is LLM argument extraction: the user's natural language sentence is passed to GPT-4o-mini, which returns the structured JSON object described in Section III-B. This step must complete first because all subsequent steps depend on having named, typed field values. Step 2 is date resolution via `FlexibleDateConverter`: relative date expressions such as "next month" or "last summer" are translated into concrete `DateTime` ranges using the system clock; this cannot run before Step 1 because it requires resolved field values, not the raw sentence. Step 3 is embedding generation: the extracted fields are concatenated into a composite string and sent to `text-embedding-3-small`, which returns a 1,536-dimensional vector. Step 4 is duplicate detection: the new asset's name and keyword set are compared against all existing records; a save is blocked if the overlap threshold is met. Step 5 is the write to `embeddings.json`: the embedding vector is persisted first. Step 6 is the write to

reminders.csv: the asset metadata row is appended only after the JSON write succeeds, preserving the invariant that every CSV row has a corresponding embedding. If any step throws an exception, the remaining steps do not execute and both storage files are left in a consistent state.

G. Semantic Search Design

Semantic search retrieves reminders based on meaning rather than exact keywords. At save time, CreateAsset generates an embedding from a combined representation of Name, Category, UserInput, Experience, and Tags, which is passed to OpenAI's text-embedding-3-small returning a 1,536-dimensional vector $\mathbf{v} \in \mathbb{R}^{1536}$.

At query time, SearchRemindersSemantic generates a query embedding $\mathbf{q}$ and compares it against each stored asset embedding $\mathbf{a}_i$ using cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{a}_i) = \frac{\mathbf{q} \cdot \mathbf{a}_i}{\|\mathbf{q}\| \cdot \|\mathbf{a}_i\|} \tag{1}$$

A threshold $\tau = 0.25$ is used for purely semantic queries to allow broader matching, and $\tau = 0.75$ within the structured SearchAssets pipeline where precision matters more. Assets with $\text{sim}(\mathbf{q}, \mathbf{a}_i) \geq \tau$ are ranked by descending score and the top-k results are returned.

H. Date Resolution

CreateAsset and SearchAssets handle two classes of temporal expression. Absolute dates in DD-MM-YYYY format are parsed with DateTime.TryParseExact. Relative expressions such as "last week", "last month", and "next summer" are handled by a ResolveRelativeDate helper derived from the system clock t_0. For a relative offset of n months into the past:

$$\begin{aligned} t_{\text{start}} &= \text{StartOfMonth}(t_0 - n \text{ months}) \\ t_{\text{end}} &= \text{EndOfMonth}(t_0 - 1 \text{ month}) \end{aligned} \tag{2}$$

Assets whose EventDate d satisfies $t_{\text{start}} \leq d \leq t_{\text{end}}$ are retained. Season names map to fixed calendar quarters.

I. Duplicate Detection

SaveAssetAsync applies a two-level guard. The first level is an exact match on (Name, Category). The second extracts significant keywords $K(s)$ from a name string s:

$$K(s) = \{w \in \text{tokens}(s) \mid |w| > 2\} \tag{3}$$

A save is blocked if $|K(s_{\text{new}}) \cap K(s_i)| \geq 2$. The threshold of two shared keywords was empirically tuned to minimise false positives while catching genuine duplicates. For example, "Vapiano" and "Vapiano restaurant Sachsenhausen" share only the single keyword "Vapiano" and pass through correctly as separate entries. By contrast, "Vapiano Italian restaurant" and "Vapiano restaurant Frankfurt" share both "Vapiano" and "restaurant", triggering the guard. Two different restaurants that happen to share only a cuisine name would not be blocked because their keyword overlap would be exactly one, which falls below the threshold.

IV. IMPLEMENTATION

A. Technology Stack

As summarised in Table II, Reminder Agent targets net10.0 and is written entirely in C#, using a focused set of Microsoft and OpenAI dependencies with no external cloud services beyond the OpenAI API. The solution uses the **Microsoft Agent Framework** (MAF), specifically the Microsoft.Extensions.AI abstraction layer and the Microsoft.Extensions.AI.OpenAI provider package. MAF provides the IChatClient interface for model-agnostic chat client wiring and the UseFunctionInvocation() middleware that automates the ReAct-style tool dispatch loop, eliminating the need to implement LLM-to-tool routing manually. **GPT-4o-mini** was chosen as the orchestration and RAG generation model because it supports structured JSON output via function calling at lower latency and cost than larger models, which is important for a conversational REPL where each user turn involves at least two model calls. The **text-embedding-3-small** model from OpenAI produces 1,536-dimensional vectors and was preferred over larger embedding models because its accuracy on short descriptive text is sufficient for the cosine thresholds used in this system, while its per-call cost is substantially lower. **CsvHelper 33.0.1** manages all CSV serialisation and deserialisation, including the custom AssetMap class and FlexibleDateConverter that resolve ambiguous date formats at read time. The **Microsoft.Extensions.DependencyInjection** container, accessed through the .NET Generic Host, wires all components as singletons through the single composition root in Program.cs, allowing each layer to be tested in isolation without service locator patterns. **Moq** and **FluentAssertions** are test-only dependencies with no runtime presence in the main application. No external database, cloud storage, or message broker is used; persistence is limited to local flat files. Configuration is handled through the .NET appsettings.json file, which contains two values under the OpenAI section: the API key and the model name. The API key is stored as a placeholder (YOUR_API_KEY_HERE) in the committed file so that no real credentials are ever checked into source control; each developer replaces this value locally before running the application. The model name defaults to gpt-4o-mini and can be changed in appsettings.json without modifying or recompiling any code, making it straightforward to switch to a different model in future. The API key can also be supplied as a command-line argument (--OpenAI:ApiKey=your-key-here), following the standard .NET configuration pipeline rather than reading directly from environment variables. This keeps the system self-contained and reproducible on any machine that has a configured appsettings.json.

B. Coding Strategy and Design Rationale

Several explicit design decisions shaped the implementation, each chosen for a specific reason rather than convenience.

1) Interface-first design: Every infrastructure component, IStorageProvider, IEmbeddingStore,

TABLE II
REMINDERAGENT TECHNOLOGY STACK

Component	Technology	Detail
Language / Runtime	C# / .NET	net10.0 TFM
Agent Framework	Microsoft Agent Framework	IChatClient, AIFunctionFactory, UseFunctionInvocation
LLM	GPT-4o-mini	Orchestration and RAG generation
Embedding Model	text-embedding-3-small	1,536-dimensional vectors
Structured Storage	CSV via CsvHelper 33.0.1	Custom AssetMap, FlexibleDateConverter
Vector Storage	JSON via System.Text.Json	Atomic write, sidecar file
Observability	FileLogger (static)	INFO/WARNING/ERROR, date-stamped logs
IDE / Build	Visual Studio / MSBuild	net10.0 target
Configuration	appsettings.json + .NET IConfiguration	API key (placeholder), model name; command-line override supported
Testing	MSTest v3 + Moq + FluentAssertions	Unit and integration tests

`IEmbeddingService`, and `ISimilarityService`, is defined as an interface before any concrete implementation. This decouples the tool layer from specific storage or embedding technology and makes unit testing precise: each test injects only the dependency under test and mocks everything else through Moq.

2) Single composition root: `Program.cs` is the single location where all services are wired together using the .NET Generic Host DI container. This rejects service locator patterns and static singletons, which hide dependencies and make tests fragile. The full list of registrations and their order is described in Section IV-C.

3) Sequential ingestion pipeline: `CreateAsset` executes the six steps described in Section III-F in strict sequence with no shared mutable state between steps. Each step receives its inputs as parameters and returns its result explicitly, so an exception in any step carries a clear message, does not corrupt the next step's inputs, and leaves both storage files in a consistent state.

4) JSON-before-CSV write ordering: The embedding vector is persisted to `embeddings.json` before the asset row is written to `reminders.csv`. This preserves the invariant that every CSV row has a corresponding embedding. Reversing the order would allow a CSV row to exist without an embedding, silently breaking semantic retrieval.

5) Tool descriptions as routing contracts: Each tool function carries a `[Description]` attribute that doubles as its routing specification, keeping the contract between the LLM and the implementation co-located. If a tool's behaviour changes, its description is updated in the same edit, preventing the common failure mode where routing logic drifts from implementation over time.

C. Dependency Injection and Agent Orchestration

All components are wired through the .NET Generic Host DI container in `Program.cs`. As shown in Algorithm 1, the registrations follow a strict order determined by construction-time dependencies. First, `OpenAIClient` is registered as a shared singleton used by both the chat client and the embedding generator. Second, `IChatClient` is registered backed by GPT-4o-mini with `UseFunctionInvocation()` middleware enabled. Third, `IEmbeddingGenerator` is registered backed by text-embedding-3-small. Fourth, `IEmbeddingStore` is registered as `JsonEmbeddingStore`. Fifth, `IStorageProvider` is registered as `CsvStorageProvider`, which receives `IEmbeddingStore` at construction this is why the embedding store must be registered first. Sixth, `IEmbeddingService` is registered as `EmbeddingService`. Seventh, `ISimilarityService` is registered as `SimilarityService`. Finally, `ReminderTools` is registered with all four dependencies (`IStorageProvider`, `IEmbeddingService`, `ISimilarityService`, and `IChatClient`) injected. Each service is registered once against its interface, keeping every class decoupled and independently testable.

Algorithm 1 Service Registration and Tool Initialisation

```
INITIALIZE services
REGISTER JsonEmbeddingStore
REGISTER CsvStorageProvider (depends on EmbeddingStore)
REGISTER ChatClient (with UseFunctionInvocation enabled)
for each method IN ReminderTools do
    REGISTER method AS AIFunction via AIFunctionFactory
end for
```

Each step of the tool dispatch cycle is managed by `UseFunctionInvocation()` middleware, which recognises a tool call, invokes the C# method, injects the result as `FunctionResultContent`, and initiates a second completion. RAG retrieval tools build a structured context string from retrieved assets and pass it to a second `GetResponseAsync` call that produces the final natural-prose response shown to the user.

D. Hybrid Retrieval Pipeline

`SearchAssets` switches between structured metadata filtering and semantic embedding search depending on the nature of the incoming request, as illustrated in Fig. 3. When a concrete filter such as a category, date range, or location name is present, the system pre-filters the asset list and sorts results by event date, which is fast and precise. When no such filter is present, a query embedding is generated and compared against every stored asset embedding using cosine similarity with threshold $\tau = 0.75$, and the top-k results by descending

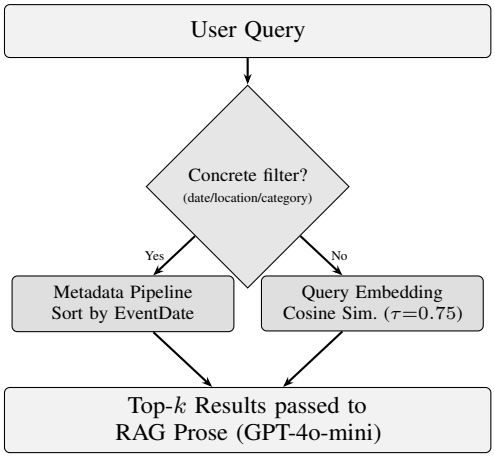

Fig. 3. Hybrid retrieval routing in `SearchAssets`. Concrete filters trigger the metadata pipeline while vague queries trigger cosine similarity over embeddings. Both paths converge to a RAG call that produces natural-language prose.

score are returned. If the semantic scan finds no result above the threshold, the system falls back to returning the most recent assets by date. Location matching is bidirectional so that a search for a city name matches the city field and a search for a country name matches the country field, handling both partial and complete location terms without requiring the user to be precise.

V. TESTING STRATEGY

A. Test Suite Structure

The test suite is organised into two tiers. The first tier consists of unit tests that verify individual components in isolation using `InMemoryEmbeddingStore` and GUID-named CSV files to keep the file system out of the picture. The second tier comprises three independent integration test sets that exercise the full tool stack against real on-disk CSV and JSON files, with no mocks in the storage layer. Test classes are grouped by functional area rather than by development phase.

B. Entity and Early-Phase Unit Tests

`AssetEntityTests` verifies the core *Asset* domain entity, which is present in both the early Semantic Kernel implementation and the final MAF-based system. It confirms that a newly constructed `Asset` instance receives a non-empty GUID, initialises its `Tags` and `Metadata` collections to non-null empty collections, and defaults `TimelineState` to *Present*. A second test assigns name, tag, and metadata values after construction and verifies that all assignments are stored and retrieved correctly.

`AssetCreationHelperTests` covers a helper class that constructed `Asset` objects from raw user input and an agent response string during the Semantic Kernel phase of the project. The helper truncated the name field to twenty characters with an ellipsis suffix when the input exceeded that length. Although this helper is no longer called in the MAF-based implementation, the tests are preserved to document that asset creation logic was actively tested during early development. The tests verify that truncation is applied correctly for long inputs and suppressed for short inputs, and that `Category` and `TimelineState` are populated with their expected default values.

Several test classes from the Semantic Kernel phase are retained in commented-out form. `ReminderPluginTests` and `ReminderPluginFormattingTests` tested the `ReminderPlugin` class, the Semantic Kernel equivalent of the current `ReminderTools`; they are commented out because `ReminderPlugin` was removed when the project migrated to MAF in Sprint 3. `StorageProviderTests` and `CsvStorageRoundTripTests` were written against an earlier version of `CsvStorageProvider` whose constructor signature changed substantially during the migration. `TimelineClassificationTests` covered a `TimelineHelper` utility class that was removed when `ReminderTools` was updated to compute timeline state inline using a date switch expression. These tests are preserved as a record of the iterative development process and to demonstrate that testing was active across all phases of the project.

C. Unit Testing: Storage and Infrastructure

The storage layer tests exercise `CsvStorageProvider` and `JsonEmbeddingStore` against real files on disk. Each test method creates a uniquely named temporary CSV file and writes a valid header row before running any assertions, ensuring that test runs never share file state. Tests cover saving and reloading a single asset with all fields populated, round-tripping multi-value fields (Tags and PhotoRefs as pipe-delimited strings), serialising and deserialising the Metadata dictionary as an embedded JSON fragment, and verifying that the atomic write pattern leaves the file in a consistent state when a simulated write failure occurs.

D. Unit Testing: Similarity and Embedding Services

`SimilarityService` tests exercise cosine scoring, threshold filtering, and top-k result ranking in isolation. As listed in Table III, six edge cases are verified covering identical, orthogonal, opposite, zero, scale-invariant, and mismatched-length vector pairs. Additional tests confirm that assets scoring below the configured threshold are excluded from results and that the top-k truncation is applied after scoring, not before. `EmbeddingService` tests verify that the service correctly concatenates the composite embedding string from Name, Category, UserInput, UserExperience, and Tags before requesting a vector from the model.

E. Unit Testing: Tool Logic and Interface Contracts

`ReminderTools` unit tests use Moq to mock all four injected dependencies (`IStorageProvider`, `IEmbeddingStore`, `IEmbeddingService`, and `ISimilarityService`), enabling focused tests for date parsing, filter logic, and duplicate detection without touching the file system or the OpenAI API. Tests cover absolute date parsing in DD-MM-YYYY format, relative expression resolution for “last month”,

"next summer", and "yesterday", timeline state reclassification at month and year boundaries, and the two-level duplicate guard with boundary cases at keyword overlap counts of one, two, and three.

Interface contract tests use .NET reflection to verify that `ISimilarityService`, `IEmbeddingService`, and `IEmbeddingStore` expose the correct method names, parameter types, return types, and nullability annotations without instantiating any concrete implementation. These tests provide a compile-independent safety net that catches interface drift in scenarios where a developer renames a method or changes a parameter type in one assembly without recompiling all dependent assemblies, a failure mode that standard compilation checks would not catch until runtime. The 14 contract tests cover all public interface members.

As shown in Fig. 4, the heaviest coverage is concentrated in `CsvStorageProvider` and `ReminderTools`, reflecting the complexity and number of code paths in those two components.

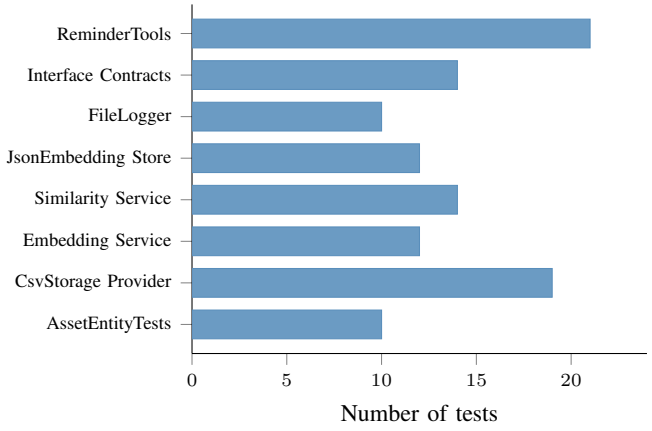

Fig. 4. Unit test distribution by component.

F. Integration Testing Approach

As illustrated in Fig. 5, the integration tests are divided into three independent sets, each instantiating real `CsvStorageProvider` and `JsonEmbeddingStore` instances pointed at uniquely named temporary files and exercising the full serialisation, file-locking, and atomic rename code paths.

Set One verified end-to-end tool chain behaviour and is contained in `ReminderTools_ToolChain_IntegrationTests.cs`. It covers `AttachPhoto` workflows and date format round-trips, including absolute and relative date expressions and season names mapped to fixed calendar quarters.

Set Two exercised CSV persistence correctness and is contained in `ReminderTools_CsvPersistence_IntegrationTests.cs`. It covers bidirectional location matching and timeline state reclassification, running at multiple simulated clock offsets to catch month and year boundary conditions.

Set Three covered the interaction between the embedding service and the CSV provider and is contained in `EmbeddingService_CsvProvider_IntegrationTests.cs`. It covers concurrent write atomicity with 20 parallel tasks, Unicode and special character serialisation through CsvHelper, and scalability by seeding 10,000 assets and confirming cosine scan latency stays under three seconds.

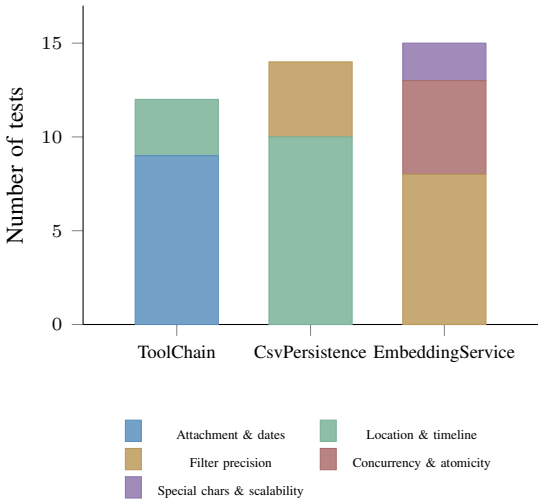

Fig. 5. Integration test distribution across three sets.

G. Validation of Critical Paths

Five defects were found and fixed before submission. The first involved relative time-period expressions being routed to the wrong tool, which was caught by date-focused tests. The second was an off-by-one error at month boundaries in `ParseDateRange`, caught by boundary condition tests. The third involved semantic scoring running when hard filters should have taken precedence, verified by filter-precision tests. The fourth was `AttachPhoto` occasionally generating invented file paths, which was fixed at the prompt level. The fifth arose when the CSV was updated but the provider had not reloaded, causing stale timeline results, and was resolved by restarting the provider between write and read in the integration tests. None of these defects would have been caught by unit tests using mocked storage.

VI. RESULTS AND EVALUATION

A. Intent Routing Reliability

All seven tool functions were exercised across a range of natural language phrasings covering different vocabulary, word order, and levels of specificity. GPT-4o-mini dispatched correctly to the intended tool in every tested case. `CreateAsset` successfully inferred location fields from implicit context, so the input "visited the Colosseum last summer" produced city=Rome, country=Italy, region=Lazio without any explicit location fields being provided. The LLM extracted this context from the `UserInput` field during argument extraction, before any storage operation took place. `GetReminders` accurately reclassified stale "Present" records as "Past" by dynamically

computing `TimelineState` at query time rather than relying on a fixed stored value, which means no data migration is needed when a reminder's date passes. Natural language count expressions such as "top 3 restaurants in Frankfurt" were handled correctly through the `topK` argument, which the LLM populated directly from the phrasing of the request. `AttachPhoto` modified only the target asset row in every test case, leaving all sibling records untouched, and correctly rejected photo file extensions outside the permitted list before attempting any write.

B. Hybrid Retrieval Performance

The two retrieval modes handle complementary request types that neither alone could serve correctly. Structured metadata filtering is fast and precise for date, location, and category requests but returns no results when the user only remembers a feeling or a description. Cosine similarity over embeddings handles the latter case but is less precise and slower when an exact date or city name is available. The hybrid design routes automatically between the two. The cosine scan over 10,000 assets completed in under three seconds on commodity hardware, while structured filtering returned results in milliseconds. As shown in Table III, the correctness of the cosine similarity implementation was verified against six mathematically defined edge cases, each chosen to test a distinct property of the formula. A brief explanation of each row follows for clarity.

Identical vectors (expected score 1.0): when the query embedding and the stored embedding are exactly the same, the cosine similarity must return 1.0, the maximum possible value. This verifies the basic correctness of the dot product and normalisation calculation.

Orthogonal vectors (expected score 0.0): two vectors at a 90-degree angle have a dot product of zero, meaning they share no directional component and are semantically unrelated. The result must be exactly 0.0.

Opposite vectors, clamped (expected score 0.0): mathematically, two opposite vectors produce a cosine similarity of -1.0. The implementation clamps the result to 0.0 because a negative similarity score has no meaningful interpretation in a retrieval context — it should not be possible to be "less than unrelated".

Zero vector (expected score 0.0): if an asset embedding is a zero vector (all elements equal to zero), the formula would produce a division by zero. The implementation must handle this gracefully by returning 0.0 rather than throwing an arithmetic exception.

Scale-invariant (expected score 1.0): cosine similarity measures direction, not magnitude. Multiplying a vector by any positive scalar should not change the similarity score. This test confirms that scaling an embedding (e.g. because of different normalisation) does not affect retrieval ranking.

Mismatched lengths (expected: Exception): comparing two embeddings of different dimensions is undefined behaviour. The implementation must detect this and throw an exception rather than silently producing an incorrect result. This is particularly important as a safety net for future model upgrades that change embedding dimensionality.

The lower threshold of $\tau = 0.25$ for the semantic tool is intentional. When a user asks a vague question such as "that mountain place I liked", a higher threshold would return no results because the cosine score between a loosely described memory and its stored embedding may be moderate rather than high. Accepting results down to 0.25 trades some precision for significantly better recall in the feeling-based query case. Within `SearchAssets`, where the user has already provided a concrete filter, the threshold is raised to $\tau = 0.75$ because a higher-confidence semantic match is expected when the request already contains a specific term.

TABLE III
COSINE SIMILARITY EDGE CASE VERIFICATION

Test Scenario	Expected	Verified By
Identical vectors	1.0	`CosineSimilarity_IdenticalVectors_ReturnsOne`
Orthogonal vectors	0.0	`CosineSimilarity_OrthogonalVectors_ReturnsZero`
Opposite (clamped)	0.0	`CosineSimilarity_OppositeVectors_ClampedToZero`
Zero vector	0.0	`CosineSimilarity_ZeroVector_DoesNotThrow`
Scale-invariant	1.0	`CosineSimilarity_ScaledVector_ReturnsSameScore`
Mismatched lengths	Exception	`CosineSimilarity_MismatchedLengths_Throws`

C. Storage Consistency

The CSV and JSON stores maintained full data consistency across all tested conditions. Full-fidelity round-trips were verified for all field types including multi-value lists (Tags and PhotoRefs), the metadata dictionary, and 1,536-dimensional embedding vectors with no floating-point precision loss across the JSON serialise-deserialise cycle. The concurrent-write test with 20 parallel tasks confirmed zero corrupted entries, validating the `SemaphoreSlim` gate and atomic rename pattern working together to prevent partial writes under load. The file-lock detection test confirmed that when the CSV is held open by Excel, the system raises a descriptive error and exits cleanly rather than attempting a partial write that would corrupt the file. The backup mechanism was also exercised: after a simulated write failure, the backup copy restored the last known good state with no data loss beyond the failed write itself. A separate test confirmed that the `JsonEmbeddingStore` correctly handles an empty or newly initialised `embeddings.json` file without throwing on the first read, which is a common failure mode in sidecar-file designs.

D. Interface Contract Compliance

Interface contract tests verified that `ISimilarityService`, `IEmbeddingService`, and `IEmbeddingStore` expose the correct method names, parameter types, and return types using .NET reflection. These tests run without instantiating any concrete implementation, providing a compile-independent safety net that catches breaking changes without requiring a rebuild of dependent projects. The reflection-based approach was chosen specifically because it detects interface drift in scenarios where a developer renames a

method or changes a parameter type in one assembly without recompiling all dependent assemblies, a failure mode that standard compilation checks would not catch until runtime. The 14 contract tests cover all public interface members and verify both the method signature and the nullability annotations, ensuring that optional parameters remain optional and required parameters remain required across refactoring cycles.

VII. DISCUSSION

A. Limitations

The system has three principal limitations that bound the scope of the results. First, the cosine scan is linear in the number of stored assets. Scoring 10,000 assets takes under three seconds, but at 100,000 or more an approximate nearest-neighbour index such as HNSW would be required. Second, the storage design is single-user. The `SemaphoreSlim` gate serialises writes within a single process but does not prevent corruption from concurrent access by separate processes or users. Third, without a geocoding API, location inference depends entirely on GPT-4o-mini's world knowledge, so obscure or newly opened venues may not resolve correctly to city and country fields.

B. Design Trade-offs

CSV was chosen deliberately over a database. The stored file opens instantly in Excel, requiring no server or connection string, and a supervisor can inspect or edit the data directly without installing any software. The real cost is that each write operation rewrites the entire file rather than updating a single record, and concurrent access requires an explicit lock rather than the row-level locking a database provides natively. In practice neither concern arises for a personal tool with a few hundred records, and both are handled cleanly using `SemaphoreSlim(1,1)` and the atomic rename pattern without adding any external dependencies.

Storing embeddings in a separate JSON sidecar rather than CSV columns was driven partly by CsvHelper's serialisation constraints and partly by a genuine architectural benefit. If the embedding model is upgraded to one with different dimensionality, the JSON store can be deleted and regenerated from the existing CSV records without touching the human-readable asset data. The two stores are loosely coupled through the shared GUID key, which appears as the primary identifier in both files.

The GPT-4o-mini model choice was driven by cost and latency. GPT-4o-mini is substantially cheaper per token and responds faster than larger models, which matters for a conversational application where each user turn requires at least two model calls. For the intent routing and argument extraction task, the smaller model performed correctly on all tested inputs, and the richer language understanding of a larger model was not needed for structured extraction from short natural-language sentences.

C. Future Directions

The most immediate improvement would be replacing keyword overlap with embedding distance for duplicate detection. Two assets that describe the same place using different words would currently both be saved, and using the cosine similarity of their embeddings as a duplicate signal would catch these cases automatically, though a suitable threshold would need to be calibrated empirically to avoid false positives between genuinely different assets in the same category.

Accepting images at creation time and generating descriptive embeddings from them would extend the system to visual memories such as photos of menus, business cards, and scenery. This would require a multimodal embedding model rather than text-embedding-3-small, and the storage model would need to accommodate both text and image embedding vectors per asset.

Access control is a natural next step since the system currently supports only a single user. Adding per-user data isolation, either through separate file directories or a lightweight authentication layer, would open it to shared or family use without requiring a full database migration. Proactive delivery could surface reminders automatically based on the user's current location or the proximity of a saved event date, removing the need to query explicitly. At scale, the in-memory cosine scan would need to be replaced with an approximate nearest-neighbour index such as HNSW, and a Blazor or .NET MAUI front-end would make the system accessible beyond the command line. The `IStorageProvider` and `IEmbeddingStore` interfaces are already in place to support all of these transitions without changing the tool layer or the orchestration logic above it.

VIII. CONCLUSION

Reminder Agent demonstrates that conversational capture and retrieval of personal assets is achievable without a database, structured forms, or exact keyword recall. The hybrid retrieval design proved necessary because structured metadata filtering alone missed feeling-based requests while semantic search alone could not handle date or location constraints efficiently. The file-based persistence layer with atomic write guarantees and `SemaphoreSlim` concurrency control maintained full data consistency under all tested conditions, demonstrating that a database is not required for personal-use asset management. The test suite, run against real files on disk rather than mocked storage, confirmed correct behaviour and surfaced five correctness defects that would have remained hidden in a purely mocked test environment. The principal limitation is the linear scalability of the cosine scan, which bounds the approach to personal-scale collections without an approximate nearest-neighbour index. The architectural interfaces are already in place to support future upgrades to the storage and retrieval layers without changing the tool logic or orchestration above them.

ACKNOWLEDGMENT

The authors thank Prof. Damir Dobric for supervising this project throughout the Software Engineering module at Frankfurt University of Applied Sciences. The Microsoft Agent Framework documentation and the OpenAI API platform were essential resources during development.

REFERENCES

[1] C. Khoo, B. Luyt, E. Caroline, O. Jamila, L. Hui-Hui, and Y. Sally, "How users organize electronic files on their workstations in the office environment: A preliminary study of personal information organization behaviour," *Inf. Res. Int. Electron. J.*, vol. 11, Jan. 2007.

[2] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., Curran Associates, Inc., 2020, pp. 9459–9474.

[3] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," Mar. 10, 2023, arXiv:2210.03629.

[4] "AI apps for .NET developers — .NET | Microsoft Learn," Accessed: Mar. 27, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/ai/

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," May 24, 2019, arXiv:1810.04805.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Aug. 27, 2019, arXiv:1908.10084.

[7] OpenAI, "New embedding models and API updates," Accessed: Mar. 27, 2026. [Online]. Available: https://openai.com/index/new-embedding-models-and-api-updates/

[8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," in *Proc. 2024 Conf. North American Chapter of the ACL*, Jun. 2024, pp. 7036–7050.

[9] "LLM Function Calling: Complete Implementation Guide," Accessed: Mar. 27, 2026. [Online]. Available: https://blog.premai.io/llm-function-calling-complete-implementation-guide-2026/

[10] D. Skeen and M. Stonebraker, "A Formal Model of Crash Recovery in a Distributed System," *IEEE Trans. Softw. Eng.*, vol. SE-9, no. 3, pp. 219–228, May 1983.